

List Methods

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list (shallow copy)
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any Iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

List Methods Examples

```
x = [1, 2]
x.append(3)
# x will become [1, 2, 3]
x.extend([4, 5])
# x will become [1, 2, 3, 4, 5]
x.pop(-1)
# pop the last element of x
# x will become [1, 2, 3, 4]
x.insert(3, 1)
# insert 1 at the 3rd index
# x will become [1, 2, 3, 1, 4]
a = x.copy()
# a is a shallow copy of x, which is [1, 2, 3, 1, 4]
b = x.count(1)
# count() will return the number of 1s in x, so b will be 2
c = x.index(1)
# index() will return the first index of 1 in x, so c will be 0
x.remove(1)
# remove the leftmost element that have a value of 1
# x will become [2, 3, 1, 4]
x.sort()
# sort the values of x
# x will become [1, 2, 3, 4]
x.reverse()
# reverse the order of x
# x will become [4, 3, 2, 1]
x.clear()
# x will become []
```

Set Operations

Operation	Equivalent	Result
len(s)		number of elements in set s (cardinality)
x in s		test x for membership in s
x not in s		test x for non-membership in s
s.issubset(t)	s <= t	test whether every element in s is in t
s.issuperset(t)	s >= t	test whether every element in t is in s
s.union(t)	s t	new set with elements from both s and t
s.intersection(t)	s & t	new set with elements common to s and t
s.difference(t)	s - t	new set with elements in s but not in t
s.symmetric_difference(t)	s ^ t	new set with elements in either s or t but not both
s.copy()		new set with a shallow copy of s

File Methods

Method	Description
close()	Closes the file
detach()	Returns the separated raw stream from the buffer
fileno()	Returns a number that represents the stream, from the operating system's perspective
flush()	Flushes the internal buffer
isatty()	Returns whether the file stream is interactive or not
read()	Returns the file content
readable()	Returns whether the file stream can be read or not
readline()	Returns one line from the file
readlines()	Returns a list of lines from the file
seek()	Change the file position
seekable()	Returns whether the file allows us to change the file position
tell()	Returns the current file position
truncate()	Resizes the file to a specified size
writable()	Returns whether the file can be written to or not
write()	Writes the specified string to the file
writelines()	Writes a list of strings to the file

Reading Files Examples

```
with open('foo.txt', 'r') as f:
    first_line = f.readline()
    for a_line in f:
        print(a_line.rstrip('\n'))
```

```
Hello World
Let's learn Python! (Inside foo.txt)
Python is fun!!!
```

```
Let's learn Python! Program output
Python is fun!!!
```

Note:
Reading files is like a pointer. After a part of the file is being read, it won't be repeated again.

Writing to Files Examples

```
with open('foo.txt', 'w') as f:
    f.write('Hello World\n')
    f.write('Python is fun!')
```

```
Hello World (Inside foo.txt)
Python is fun!
```

Convert file to 2D list

```
def readmap(filename):
    f = open(filename)
    m = []
    for line in f:
        m.append(list(line.rstrip('\n')))
    return m
```

Tuple Methods

Method	Description
count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found

String Comparison

If you compare strings that contain the same substring, such as Apple and ApplePie, then the longer string is considered larger

Python string comparison compares the characters in both strings one by one. When different characters are found, then their Unicode code point values are compared. The character with the lower Unicode value is considered to be smaller.

Important Things

In a dictionary in Python, values can be any data type, but keys cannot be list, dictionary, and set (unhashable type)

In string slicing, the format is [start : stop : step]. Stop is **always** exclusive. If step is 1 (default value), the default value of start will be 0. If step is set to be -1, the default value of start will change to -1.

ord('A') = 65 chr(65) = 'A'

In set operation, |, &, and ^ are associative. On the other words. ((set1 ^ set2) ^ set3) = (set1 ^ (set2 ^ set3))

Conversion must be primitive to primitive, or iterables to iterables

Map and Filter

Syntax: map(fun, iter)

Parameters:

- fun: It is a function to which map passes each element of given iterable.
- iter: iterable object to be mapped.

Note: map function is only shallow map

Syntax: filter(function, sequence)

Parameters:

- function: function that tests if each element of a sequence is true or not.
- sequence: sequence which needs to be filtered, it can be sets, lists, tuples, or containers of any iterators.

Note: map and filter will return an object. This object is consumable, so you can access them once. To make it permanent, you can change its type first by using list() or tuple().

Image Processing

Using the imageio package to read in a photo as a 3D

```
>>> import imageio
>>> import matplotlib.pyplot as plt
>>> pic = imageio.imread('green hair girl.jpg')
>>> len(pic)
900
>>> len(pic[0])
517
>>> len(pic[0][0])
3
>>> plt.imshow(pic)
<matplotlib.image.AxesImage object at ...
>>> plt.show()
```



String Methods

Method	Description
capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
expandtabs()	Sets the tab size of the string
find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
format_map()	Formats specified values in a string
index()	Searches the string for a specified value and returns the position of where it was found
isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isascii()	Returns True if all characters in the string are ascii characters
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
istitle()	Returns True if the string follows the rules of a title
isupper()	Returns True if all characters in the string are upper case
join()	Converts the elements of an iterable into a string
ljust()	Returns a left justified version of the string
lower()	Converts a string into lower case
lstrip()	Returns a left trim version of the string

maketrans()	Returns a translation table to be used in translations
partition()	Returns a tuple where the string is parted into three parts
replace()	Returns a string where a specified value is replaced with a specified value
rfind()	Searches the string for a specified value and returns the last position of where it was found
rindex()	Searches the string for a specified value and returns the last position of where it was found
rjust()	Returns a right justified version of the string
rpartition()	Returns a tuple where the string is parted into three parts
rsplit()	Splits the string at the specified separator, and returns a list
rstrip()	Returns a right trim version of the string
split()	Splits the string at the specified separator, and returns a list
splitlines()	Splits the string at line breaks and returns a list
startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string
swapcase()	Swaps cases, lower case becomes upper case and vice versa
title()	Converts the first character of each word to upper case
translate()	Returns a translated string
upper()	Converts a string into upper case
zfill()	Fills the string with a specified number of 0 values at the beginning

Reduce

```
def reduce(f, seq):
    if not seq:
        return seq
    first = seq[0]
    for i in seq[1:]:
        first = f(first, i)
    return first
```

Written by Jansen Ken Pegrasio

Dictionary Methods

Method	Description
<code>clear()</code>	Removes all the elements from the dictionary
<code>copy()</code>	Returns a copy of the dictionary
<code>fromkeys()</code>	Returns a dictionary with the specified keys and value
<code>get()</code>	Returns the value of the specified key
<code>items()</code>	Returns a list containing a tuple for each key value pair
<code>keys()</code>	Returns a list containing the dictionary's keys
<code>pop()</code>	Removes the element with the specified key
<code>popitem()</code>	Removes the last inserted key-value pair
<code>setdefault()</code>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<code>update()</code>	Updates the dictionary with the specified key-value pairs
<code>values()</code>	Returns a list of all the values in the dictionary

Dictionary Methods Examples

```
ddict = dict({'a', 1}, {'b', 2}, {'c', 3})
```

```
ddict.update({'d': 4})
```

```
x = ddict.get('a')
```

```
# x will be equal to 1, get() is similar to []
```

```
keys = ddict.keys()
```

```
for key in keys:
```

```
    print(key, end = ' ')
```

```
print()
```

```
# This will print: a b c d
```

```
values = ddict.values()
```

```
for value in values:
```

```
    print(value, end = ' ')
```

```
print()
```

```
# This will print: 1 2 3 4
```

```
ddict.pop('d')
```

```
items = ddict.items()
```

```
for key, value in items:
```

```
    print(key, value)
```

```
'''
```

```
This will print:
```

```
a 1
```

```
b 2
```

```
c 3
```

```
'''
```

Deep Map and Deep Copy

```
def deepMap(f, seq):
    if seq == []:
        return seq
    if type(seq) != list:
        return f(seq)
    return [deepMap(f, seq[0])] + deepMap(f, seq[1:])
```

```
def deepCopy(seq):
    return deepMap(lambda x:x, seq)
```

Flatten

```
def flatten(seq):
    if seq == []:
        return seq
    if type(seq) != list:
        return [seq]
    return flatten(seq[0]) + flatten(seq[1:])
```

Class

```
class BankAccount():
    def __init__(self, name, balance, interestrate):
        self.name = name
        self.balance = balance
        self.ir = interestrate
    def withdraw(self, name, amount):
        if name != self.name:
            print("You are not authorized for this account")
            return
        if self.balance < amount:
            print(f"Money not enough! You do not have ${amount}")
            return 0
        else:
            self.balance -= amount
            return amount
    def deposit(self, amount):
        self.balance += amount
    def oneYearHasPass(self):
        self.balance *= 1 + self.ir
    def showBalance(self):
        print(f'Your balance is ${self.balance}')
```

Inheritance

```
class Vehicle:
    def __init__(self, pos):
        self.pos = pos
        self.velocity = (0,0)
```

```
class Lorry(Vehicle):
    def __init__(self, pos):
        super().__init__(pos)
        self.cargo = []
```

```
class Cannon:
    def __init__(self):
        self.numAmmo = 0
```

```
class Tank(Vehicle, Cannon):
    def __init__(self, pos):
        Vehicle.__init__(self, pos)
        Cannon.__init__(self)
```

File Opening Modes

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.